Advanced Programming for Scientists

Lecture 6

Real-Time Execution and Embedded Control

Arduino case study

v1.0

Prof. Mateusz Bawaj

A.D. 1308 unipg
UNIVERSITÀ DEGLI STUDI DI PERUGIA

Motivation: Real-time in scientific systems

Many experiments require deterministic timing, not just fast execution

Examples:

- feedback control loops (optics, interferometers, PID loops)
- data acquisition synchronized with physical processes
- triggering events (laser pulses, detectors)

Key distinction:

- High-performance computing (HPC) → throughput
- Real-time systems → latency + determinism (predictable and repeatable behaviour)

In GW detectors: control loops must run at kHz rates with strict timing constraints

What is a Real-Time System?

A system where **correctness depends on timing**

Two regimes:

- **Soft real-time**: occasional delay acceptable (e.g. data logging)
- **Hard real-time**: missing deadline = system failure (e.g. control loop instability)

Key metrics:

- latency
- jitter
- determinism

Arduino as a Minimal Real-Time Platform

Arduino = **microcontroller (MCU)**, not a full computer

No operating system (by default)

Single compiled program in C++ runs:

```
void setup() { }
void loop() { }
```

Characteristics:

- single process
- no memory protection
- no scheduler (unless implemented manually)

Good to understand fundamentals of real-time execution

Single Process Computing

Arduino executes:

- one instruction at a time
- in a **single thread of execution**

Implications:

- no parallel processes
- no preemption (unless interrupts used)
- programmer controls scheduling

The `loop()` as a Control Cycle

Typical structure:

```
void loop() {
    readSensors();
    computeControl();
    writeActuators();
}
```

Equivalent to:

- discrete-time simulation
- control system iteration

This is essentially a real-time numerical integrator

Limitations of Pure Loop Execution

Blocking functions break timing:

```
delay(1000);  // BAD for real-time
```

Problems:

- unpredictable latency
- missed events
- no responsiveness

Need asynchronous mechanisms → **interrupts**

Interrupts: Concept

Interrupt = **hardware-triggered function call**

Temporarily stops main execution

Analogy:

- like an **event-driven callback**
- similar to:
  - signal handling in Linux
  - hardware triggers in DAQ systems

Interrupt Mechanism

Flow:

1)Event occurs (timer, pin change)

2)CPU pauses current execution and saves its state

3)Executes **ISR (Interrupt Service Routine)**

4)Returns to previous state

```
void ISR_function() {
    // quick, minimal code
}
```

Key Rules for Interrupts

Keep ISR:

- **short**
- **deterministic** (of the same length and repetible)

Avoid:

- **delays**
- **dynamic memory**
- **complex logic**

Use volatile variables:

```
volatile int flag = 0;
```

Example: Timer Interrupt

```
volatile bool tick = false;

ISR(TIMER1_COMPA_vect) {
    tick = true;
}

void loop() {
    if (tick) {
        tick = false;
        controlStep();
    }
}
```

Guarantees fixed frequency execution

Equivalent to:

- fixed time-step simulation
- synchronous data acquisition

From Loop to Scheduler

Problem:

- multiple tasks needed:
  - sensor reading
  - communication
  - control

Solution:

- implement a **simple scheduler**

Context switch

Preemptive vs. Cooperative Multitasking

- Preemptive (Modern): The operating system forcibly interrupts tasks to manage CPU time, preventing any single program from locking up the system.
- Cooperative (Legacy): Tasks voluntarily yield control of the CPU when they reach a stopping point. If a task fails to yield, the entire system can hang.

Similar to: event loop in Python, simulation kernels

Simple Task Scheduler (Concept)

We define:

- multiple tasks
- execution order
- switching logic

```
typedef void (*Task)();
```

Example: Two-Task Scheduler

```
void task1() {
    // e.g. sensor read
}
void task2() {
    // e.g. control computation
}
void loop() {
    task1();
    task2();
}
```

- deterministic order
- no true concurrency

Simple context change

```
Task tasks[] = {task1, task2, task3};
int current = 0;

void loop() {
    tasks[current]();
    current = (current + 1) % 3;
```

- simple context switching
- **not equal time distribution!**

Starvation problem

Starvation = a situation where a task never gets CPU time because other tasks continuously take priority

The task is:

- not blocked
- not finished
- but **never scheduled**

It “waits forever”

Starvation problem solution

```
unsigned long lastTime = 0;
const int dt = 10;

void loop() {
    if (millis() - lastTime >= dt) {
        lastTime = millis();
        tasks[current]();
        current = (current + 1) % 3;
    }
}
```

Starvation problem solution

```
unsigned long lastTime = 0;
const int dt = 10;

void loop() {
    if (millis() - lastTime >= dt) {
        lastTime = millis();
        tasks[current]();
        current = (current + 1) % 3;
    }
}
```

Will it help?

Scheduler + Interrupt

Round Robin (RR) scheduling is a preemptive, cyclic CPU scheduling algorithm where each process is assigned a fixed time unit, known as a "time quantum" or "slice," in circular order. It ensures fairness, allowing all tasks an equal share of CPU time without starvation, making it ideal for time-sharing systems.

To avoid starvation we need the **interrupt**!

Comparison with Modern Systems

Feature	Arduino	Linux / HPC
Processes	single	many
Scheduling	manual	OS scheduler
Timing	deterministic	best-effort
Interrupts	direct	abstracted

Key Takeaways

Real-time ≠ fast, but predictable

Arduino shows:

- single-process execution
- explicit scheduling
- interrupt-driven control

You can build:

- deterministic control loops
- precisely timed event driven execution